Chủ đề 04: Kiến trúc Clean Architecture & DI

TRƯỜNG ĐẠI HỌC BÁCH KHOA — KHOA ĐIỆN TỬ VÀ TRÍ TUỆ NHÂN TẠO

Báo cáo Tuần 01 — Môn: Lập trình Đa nền tảng

SVTH: Huỳnh Thiện Lộc – **MSSV:** 106230052
GVHD: TS. Nguyễn Duy Nhật Viễn

Đà Nẵng, 2026

Nội dung báo cáo

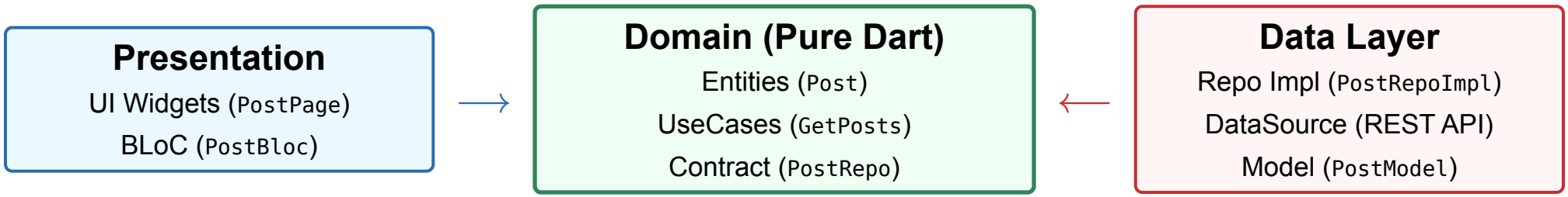
1. Phần I: Kiến Trúc 3 Tầng & Nguyên Lý DIP
   - Kiến trúc 3 tầng Clean Architecture
   - Nguyên lý Đảo ngược Phụ thuộc (DIP)
2. Phần II: Dependency Injection (DI) & Khởi Tạo Phụ Thuộc
   - Service Locator với get_it
   - Khởi tạo hệ thống & MultiBlocProvider
3. Phần III: Demo Thực Tế & Unit Test Tầng Domain
   - Luồng dữ liệu Demo (Data Flow)
   - Unit Test Tầng Domain với Mocktail
   - Tài liệu tham khảo chi tiết

Phân công công việc (WBS)

TT	HỌ VÀ TÊN	NỘI DUNG ĐẢM NHẬN	TỶ LỆ
1	Huỳnh Thiện Lộc (106230052)	Thiết kế Clean Architecture 3 tầng, cấu hình DI (`get_it`), lập trình BLoC, viết Unit Test Mocktail.	100%

Phần I: Kiến Trúc 3 Tầng & Nguyên Lý DIP

Kiến trúc 3 tầng Clean Architecture



- **Presentation**: Chỉ tương tác với UseCase & Entity, **không import tầng Data**.
- **Domain (Trái tim)**: Thuần Dart 100%, chạy Unit Test trong vài mili-giây.
- **Data**: Triển khai truy xuất API thật và hiện thực interface từ Domain.

Nguyên lý Đảo ngược Phụ thuộc (DIP)

Truyền thống (Khớp nối chặt)

`Presentation` ⟶ `RemoteDataSource`

UI phụ thuộc trực tiếp vào HTTP Client.
Đổi API hoặc thư viện mạng ⟶ Vỡ toàn bộ UI.

Áp dụng DIP (Khớp nối lỏng)

`Presentation` ⟶ `[PostRepository]` ⟵ `Data`

Cả 2 tầng cùng phụ thuộc Interface ở Domain.
Đổi REST API sang SQLite/Mock ⟶ UI giữ nguyên.

- **Bản chất**: Chiều phụ thuộc mã nguồn **ngược lại** với chiều thực thi dữ liệu.
- **Minh chứng trong code**: `post_page.dart` và `post_bloc.dart` hoàn toàn không có bất kỳ dòng import nào trỏ vào thư mục `data/`.

Phần II: Dependency Injection (DI) & Khởi Tạo Phụ Thuộc

Service Locator với get_it

- **Vai trò DI**: Tập trung khởi tạo đối tượng tại `injection_container.dart`, không để class tự `new`.

registerLazySingleton

- `http.Client` (Client mạng)
- `PostRemoteDataSource`
- `PostRepository`
- `GetPostsUseCase`

→ *Dùng 1 bản thể duy nhất để tiết kiệm RAM.*

registerFactory

- `PostBloc` (Quản lý trạng thái)

→ *Tạo instance mới mỗi lần màn hình mở.*
→ *Tự động dispose qua MultiBlocProvider.*
→ *Tránh rò rỉ bộ nhớ (Memory Leak).*

Khởi tạo hệ thống & MultiBlocProvider

- **`WidgetsFlutterBinding.ensureInitialized()`:**
  1. Bắt buộc gọi trước khi thực thi `initDependencies()` trong `main.dart`.
  2. Đảm bảo Flutter Engine & kênh BinaryMessenger native sẵn sàng trước `runApp()`.

- **`MultiBlocProvider`:**
  1. Nhúng instance `PostBloc` từ `get_it` vào Root Widget Tree.
  2. Cung cấp State cho `PostPage` thông qua `BlocBuilder<PostBloc, PostState>`.
  3. Đảm bảo đóng BLoC khi màn hình bị hủy.

Phần III: Demo Thực Tế & Unit Test Tầng Domain

Luồng dữ liệu Demo (Data Flow)


```
JSONPlaceholder API ⟶ PostRemoteDataSource ⟶ PostModel
⟶ PostRepositoryImpl ⟶ Post (Entity) ⟶ GetPostsUseCase
⟶ PostBloc (State) ⟶ PostPage (UI Trắng Đen)
```

- **Giao diện tối giản**: Danh sách bài viết trắng đen (`ListTile` + `#ID`), không rườm rà.
- **Xử lý trạng thái rõ ràng**: Đang tải (`Loading`), Danh sách bài viết (`Loaded`), Mất mạng (`Error` kèm nút Thử lại).

Unit Test Tầng Domain với Mocktail

Kịch bản 1: Thành công (Success)

`MockPostRepo` trả về `Right(List<Post>)`

⟶ `UseCase` nhận đúng danh sách

⟶ Xác minh `getPosts()` gọi đúng 1 lần

Kịch bản 2: Lỗi máy chủ (Failure)

`MockPostRepo` trả về `Left(ServerFailure)`

⟶ `UseCase` bắt lỗi an toàn qua `Either`

⟶ Không throw Exception gây crash

- Dùng `mocktail` tạo `MockPostRepository extends Mock implements PostRepository`.
- Kiểm thử cô lập 100%, không cần kết nối mạng. Kết quả: **5/5 tests pass**.

Tài liệu tham khảo chi tiết

1. **Robert C. Martin (2017)**, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*, Prentice Hall:
   - **Chương 22**: Cơ sở phân chia 3 tầng độc lập & Quy tắc phụ thuộc (Dependency Rule).
   - **Chương 11**: Nguyên lý Đảo ngược Phụ thuộc (Dependency Inversion Principle - DIP).
2. **Reso Coder (2019)**, *Flutter Clean Architecture Proposal*:
   - Kiến trúc phân thư mục, thiết kế `UseCase` & Functional Error Handling với `Either`.
3. **Felix Angelov**, *Bloc State Management Library Documentation* (https://bloclibrary.dev):
   - Quản lý trạng thái luồng sự kiện (Event-Driven) & cơ chế `MultiBlocProvider`.
4. **Thomas Burkhart**, *get_it Service Locator* (https://pub.dev/packages/get_it):
   - Vòng đời dịch vụ: `registerLazySingleton` vs `registerFactory`.